

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART 23

LeetCode Roadmap

*Easy to Medium to Hard, what actually changes between them,
and the pattern families that turn an unfamiliar problem into a
familiar one*

PART 23

LeetCode Roadmap

The same recognition skill from Part 21, aimed at a different platform

Where this Part fits

This corresponds to the roadmap's Part XXIV / Phase 23 (LeetCode). LeetCode and Codeforces (Part 22) test overlapping skills through different formats: LeetCode problems are typically untimed relative to each other, function-signature-based (no I/O parsing), and organized by company/topic tags rather than contest rounds. The underlying skill — recognizing which technique a problem needs — is exactly Part 21's, applied here to LeetCode's specific problem shapes and its Easy/Medium/Hard difficulty labels.

The gap most people hit on LeetCode isn't "I don't know what a hash map is" — it's "I know what a hash map is, but I didn't realize this problem wanted one." This Part is built entirely around closing that specific gap, organized by the pattern families that LeetCode problems draw from most heavily.

Contents of Part 23

- 1. Easy, Medium, Hard: What Actually Changes
- 2. From "Knowing DSA" to "Recognizing the Pattern"
- 3. Array, Hashing, and Two-Pointer Patterns
- 4. Stack, Binary Search, and Linked List Patterns
- 5. Tree, Graph, Backtracking, and DP Patterns
- 6. Building a Study Plan
- Part 23 Revision, Cheat Sheet, and Self-Check Questions

Chapter 1 — Easy, Medium, Hard: What Actually Changes

1.1 It's rarely about harder syntax

An Easy and a Hard problem often use the exact same C++ constructs — loops, a hash map, maybe recursion. What changes between difficulty labels is almost never the code's surface complexity; it's how many steps of reasoning stand between reading the statement and knowing what to code.

Difficulty	What typically distinguishes it	Typical technique count
Easy	Technique is usually directly named or obvious from the statement	One technique, applied directly
Medium	Requires combining two ideas, or spotting a non-obvious property (e.g. sorted-ness, monotonicity)	One technique with a twist, or two techniques combined
Hard	Requires a genuine insight, careful state design, or stacking three+ ideas correctly	Multiple techniques, or a technique used in a non-standard way

1.2 A concrete before/after

Version	Problem shape	What's actually different
Easy	Find if a target value exists in an array	Direct linear scan or hash set lookup
Medium	Find if two numbers in an array sum to a target	Same hash set idea, but now storing what to LOOK FOR (complement), not just what's present
Hard	Find if any three numbers sum to a target, without duplicate triplets	Sort first, then two pointers per fixed element, PLUS explicit duplicate-skipping logic

Notice the throughline: all three are fundamentally “find numbers satisfying a sum condition.” The hash-set idea from the Easy version doesn't disappear in the Medium version — it's reused with a twist. This is the normal shape of LeetCode's difficulty ladder: earlier ideas keep reappearing, recombined.

✓ **Practical implication for how you practice**

Don't treat Easy problems as a box to check off before “real” practice starts. The ideas they teach (hash set membership, basic two pointers, simple recursion) are the literal building blocks Medium and Hard problems recombine. Skipping them shallow leaves gaps that resurface as confusion later, not as time saved.

Quick Revision

- Difficulty differences are usually about REASONING STEPS between statement and solution, not harder syntax.
- Ideas from Easy problems reappear, recombined, inside Medium and Hard problems — they're building blocks, not a separate track.
- A Hard problem is very often 2-3 techniques you already know, stacked, plus one genuine new insight connecting them.

Chapter 2 — From "Knowing DSA" to "Recognizing the Pattern"

2.1 The gap, named precisely

“Knowing DSA” means: if told the answer uses a sliding window, you could implement it correctly. “Recognizing the pattern” means: reading a fresh, unlabeled problem statement and arriving at “sliding window” yourself, with nobody telling you. LeetCode's topic tags exist for practice, but real problems (and every interview) don't come pre-labeled.

Practicing WITH tags visible	Practicing WITHOUT tags visible
Tests: can I implement technique X correctly	Tests: can I figure out that technique X applies at all
Good for first learning a new technique	Good for building real recognition — the actual interview/contest skill
Risk: you stop reading the problem carefully once you see the tag	Forces full engagement with the statement and constraints, exactly like Part 21's process

✓ A practical training method

Learn a technique with tags visible (first exposure). Then deliberately practice a batch of MIXED, untagged problems that draw from several techniques you've already learned, forcing yourself to identify the pattern before coding. This mirrors exactly how Part 21's recognition process was meant to be trained, not just read about.

2.2 Reusing Part 21's process here

LeetCode problems skip the “read the input format” step (functions have typed signatures already), but every other step of Part 21's seven-step process applies unchanged.

Part 21 step	LeetCode-specific version
1. Understand	Read the function signature AND the full description — return type and parameter types often hint at the expected approach
2. Constraints	LeetCode gives explicit bounds (e.g. $1 \leq n \leq 10^5$) in the constraints section — use Part 21 Chapter 2's table exactly as before
3. Examples	Trace the given examples by hand, including any explanation LeetCode provides
4-7. Brute force through verify	Unchanged from Part 21

Quick Revision

- Recognizing a pattern without being told the tag is the actual transferable skill — tags are a training wheel, not the destination.
- Practice both WITH tags (learning a technique) and WITHOUT tags (testing recognition) deliberately, as two different exercises.
- Part 21's seven-step process applies directly to LeetCode problems, minus the input-format-reading step (handled by the function signature).

Chapter 3 — Array, Hashing, and Two-Pointer Patterns

3.1 The "complement lookup" family

An enormous number of array problems reduce to: for each element, is there SOME OTHER value (a complement, a duplicate, a specific difference) that I've already seen? A hash set/map turns an $O(n)$ search for that other value into $O(1)$.

Problem shape	What to store in the hash structure	Complexity
Do two elements sum to a target?	value $\rightarrow$ index, checking target - current as you go	$O(n)$
Is there a duplicate?	a set of values seen so far	$O(n)$
Longest run of consecutive integers	a set of all values, then check for run-starts (x-1 not in set)	$O(n)$
Group items by a computed key (e.g. anagram grouping)	map from computed key $\rightarrow$ list of original items	$O(n \times \text{key computation cost})$

3.2 Two pointers and sliding window

Clue	Technique	Direction
Sorted array, pair/triplet with a sum condition	two pointers from both ends	inward
Remove/partition elements in place, maintain relative order	two pointers, same direction (slow/fast)	forward
"Longest/shortest substring/subarray with property X"	sliding window, variable size	expand right, shrink left
"Maximum sum of exactly k consecutive elements"	sliding window, fixed size	slide by one each step

 The sorted-array precondition is doing a lot of work

Two-pointers-from-both-ends specifically needs sorted (or sortable-without-losing-needed-info) data — the technique relies on being able to reason “move the left pointer right to increase the sum, move the right pointer left to decrease it.” On unsorted data without that monotonic structure, this reasoning breaks and a hash-based approach is usually the right substitute.

3.3 Prefix sums, reused from Part III

“Subarray sum equals/divisible by/at most K ” problems almost always want prefix sums, frequently combined with a hash map of prefix-value $\rightarrow$ earliest (or count of) index — exactly Part 21 Chapter 6's worked walkthrough, which is itself a very common LeetCode problem shape.

Practice — Chapter 3

Problem 3.1 — Identify the pattern (no coding required) [Level 1]

For each, name the pattern family from this chapter: (a) 'find the length of the longest substring with at most two distinct characters'; (b) 'find if there exist two numbers whose difference equals k '; (c) 'find the maximum sum subarray of a fixed size k '.

Hint 1: Match each to a row in the tables above by its clue phrase.

Approach: (a) sliding window, variable size. (b) hashing (complement lookup, difference instead of sum). (c) sliding window, fixed size.

Complexity: Recognition itself is $O(1)$ once trained.

Chapter 4 — Stack, Binary Search, and Linked List Patterns

4.1 Stack-shaped problems

Clue	Technique (Part IX)
Matching/nesting (brackets, tags)	plain stack
"Next/previous greater or smaller element"	monotonic stack
Evaluate an expression with operators and parentheses	two stacks (values + operators), or postfix conversion first
Undo / most-recent-action semantics	plain stack

4.2 Binary search beyond a sorted array

LeetCode leans heavily on “binary search on the answer” (Part VI) — the array itself doesn't need to be sorted, only the underlying YES/NO condition needs to be monotonic in the candidate answer.

Clue	What's actually being binary-searched
"Minimum X such that condition holds"	X itself — classic binary search on the answer
"Find peak element" in an unsorted-looking array	the array still has a LOCAL monotonic structure around the peak, even if not globally sorted
Search in a rotated sorted array	one half is always properly sorted — identify which half, then recurse/iterate into it

⚠️ "The array isn't sorted" doesn't rule out binary search

This is one of the most common recognition misses on LeetCode. Check specifically whether a YES/NO or comparison condition is monotonic in the candidate answer — that's the actual requirement, not global sortedness of the input array.

4.3 Linked list patterns

Clue	Technique (Part VIII)
Detect a cycle, or find the middle in one pass	slow/fast (two-pointer) traversal
Reverse all or part of a list	iterative pointer-rewiring (prev/curr/next), or recursion
Merge two sorted lists	two-pointer merge, same idea as merge sort's combine step (Part VI)
Remove the k-th node from the end	two pointers, one started k steps ahead

Quick Revision

- Monotonic stack problems are almost always signaled by 'next/previous greater/smaller element' phrasing.
- Binary search on LeetCode is often 'on the answer', not on a sorted array — check for a monotonic YES/NO condition instead of assuming sortedness is required.
- Slow/fast pointers solve most single-pass linked-list structural problems (cycle detection, middle, k-th from end).

Practice — Chapter 4

Problem 4.1 — Identify the pattern (no coding required) [Level 1]

Name the pattern: 'Given a mountain-shaped array (strictly increasing then strictly decreasing), find the peak index in better than $O(n)$.'

Hint 1: The array is locally monotonic on both sides of the peak, even though it isn't globally sorted.

Approach: Binary search: compare $a[mid]$ to $a[mid+1]$; if increasing, the peak is to the right; if decreasing, the peak is at or to the left.

Complexity: $O(\log n)$, versus $O(n)$ for a linear scan.

Chapter 5 — Tree, Graph, Backtracking, and DP Patterns

5.1 Tree patterns, reused from Part XI

Clue	Technique
"Path from root to leaf", path sum	DFS passing state DOWN through parameters
Height, diameter, balanced check, subtree sum	DFS combining children's results UP through return values (postorder)
"Level by level", "right side view", zigzag	BFS (level order)
Lowest common ancestor	recursive search reporting what was found below (Part XI Ch. 7) or binary lifting (Part XVIII) if queries repeat heavily
Validate a BST	pass a valid (lo, hi) range down, never just compare immediate children

5.2 Graph patterns, reused from Part XIII

Clue	Technique
"Minimum number of steps/moves" on a grid or implicit graph	BFS — the grid IS the graph, cells are nodes
"Number of islands" / connected groups in a grid	DFS or BFS flood-fill, or DSU
Course prerequisites / build order	topological sort
"Can you reach X from Y"	DFS/BFS reachability

✓ Grids are graphs in disguise

A huge fraction of LeetCode's "matrix/grid" problems are graph problems where each cell is a node and adjacency is up/down/left/right — recognizing this immediately unlocks the entire graph toolkit (Part XIII) for what looks like a 2D-array problem.

5.3 Backtracking patterns, reused from Part X

Clue	Technique
"All possible subsets/combinations/permutations"	include/exclude or start-index backtracking (Part X, Ch. 5-6)
"All valid arrangements satisfying constraint X"	backtracking with pruning (Part X, Ch. 7) — check validity before recursing
Generate all valid parenthesizations of length n	backtracking tracking open/close counts, pruning invalid partial strings early

5.4 DP patterns, reused from Part XV

Clue	Technique
"Number of distinct ways to..."	counting DP — sum over valid transitions
"Minimum/maximum cost to..."	optimization DP — min/max over valid transitions
Decisions about including/excluding items with a capacity limit	knapsack-family DP
Two strings compared/transformed into each other	2D string DP (edit distance, LCS family)
Small n (≤ 20), "which subset has been used"	bitmask DP (Part 19)

📌 Quick Revision

- Grid/matrix problems are frequently graph problems in disguise — recognizing this unlocks BFS/DFS/DSU for what looks like array manipulation.
- Tree problems split cleanly into 'push state down' (paths) vs. 'pull state up' (metrics) — identify which direction before writing the recursion.
- 'Number of ways' signals counting DP; 'minimum/maximum cost' signals optimization DP — both share the same state/transition/base-case design process from Part XV.

Practice — Chapter 5

Problem 5.1 — Identify the pattern (no coding required) [Level 2]

Name the pattern: 'Given a grid of 1s (land) and 0s (water), count the number of distinct islands, where an island is a maximal group of connected 1s (up/down/left/right).'

Hint 1: The grid is a graph; 'maximal connected group' is a connectivity question.

Approach: DFS or BFS flood-fill from each unvisited land cell, marking the whole connected group visited; count how many times a fresh flood-fill starts. Equivalently, DSU merging adjacent land cells.

Complexity: $O(\text{rows} \times \text{cols})$.

Chapter 6 — Building a Study Plan

6.1 Order of attack

Stage	Focus	Roughly how much practice
1. Foundation	Array/hashing, two pointers, sliding window, basic stack (Chapters 3-4)	Until these feel automatic, not just doable
2. Structure	Binary search variants, linked lists, trees, basic graphs (Chapters 4-5)	Enough to recognize each WITHOUT a tag
3. Combination	Backtracking, DP (Chapter 5)	The largest time investment — these take the longest to build real intuition for
4. Integration	Mixed, untagged problem sets pulling from all of the above	Ongoing — this is where recognition actually solidifies

6.2 A practice rhythm that builds recognition, not just completion

Do this	Not this
Solve a new problem, THEN look at its tag/editorial to confirm your reasoning	Look at the tag before attempting, every time
Revisit problems you solved weeks ago, untimed, to check retention	Solve a problem once and never return to it
After finishing, ask: what was the clue phrase that should have tipped me off?	Move on immediately without extracting the reusable insight
Mix problem types within a single practice session once foundations are solid	Always practice one tag/topic in isolation, forever

✓ This is the same discipline as Part 22's upsolving process

Attempt genuinely, extract the reusable insight afterward, revisit later to check it stuck — the discipline that builds Codeforces rating and the discipline that builds LeetCode fluency are the same discipline, aimed at differently-shaped problems.

6.3 What "done" looks like for a topic

Not “I solved N problems tagged X.” Instead: given a fresh, untagged Medium-difficulty problem that uses technique X, you arrive at the right approach within the first few minutes of reading it, without needing to see a hint.

** Quick Revision**

- Build foundation techniques to automatic fluency before moving to combination-heavy topics like backtracking and DP.
- Attempt before checking tags; revisit old problems untimed to check retention; extract the clue phrase that should have tipped you off.
- 'Done' with a topic means recognizing it unprompted on a fresh problem, not a solved-problem count.

Practice — Chapter 6

Problem 6.1 — Audit your own current practice habits [Level 1]

Using the table in 6.2, honestly assess which column better describes your current practice habits, and write one specific change to make.

Hint 1: Be specific — 'practice more' isn't a habit change; 'attempt 3 untagged mixed problems every session before checking any tags' is.

Approach: Self-reflection exercise; the value is in specificity, not in a particular right answer.

Complexity: N/A.

Part 23 — Master Revision

Pattern Quick Reference

Clue phrase	Pattern
"two numbers/elements summing/differing to X"	hashing (complement lookup)
Sorted array, pair/triplet with a sum condition	two pointers, inward
"Longest/shortest substring/subarray with property X"	sliding window
"Subarray sum equals/divisible by K"	prefix sums (+ hash map)
"Next/previous greater/smaller element"	monotonic stack
"Minimum X such that condition holds"	binary search on the answer
Cycle/middle/k-th-from-end in a linked list	slow/fast two pointers
Path from root to leaf	DFS, state passed DOWN
Height/diameter/balanced/subtree sum	DFS, state combined UP (postorder)
Level by level / right side view	BFS (level order)
Grid/matrix connectivity or flood-fill	BFS/DFS/DSU — the grid is a graph
Course prerequisites / build order	topological sort
"All possible subsets/permutations/combinations"	backtracking
"Number of distinct ways"	counting DP
"Minimum/maximum cost to..."	optimization DP
Small n (≤ 20), "which subset used"	bitmask DP

Must Remember

Quick Revision

- Difficulty labels track reasoning steps, not syntax complexity — Hard problems usually recombine several ideas you already know.
- Practice both WITH tags (learning) and WITHOUT tags (testing real recognition) as distinct, deliberate exercises.
- 'The array isn't sorted' does not rule out binary search — check for a monotonic condition in the candidate answer instead.
- Grid/matrix problems are very often graph problems in disguise.
- 'Done' with a topic means unprompted recognition on a fresh problem, not a solved-count.

Common Mistakes Recap

Mistake	Fix
Treating Easy problems as skippable once 'basics are known'	Their ideas are literal building blocks for Medium/Hard — shallow gaps resurface as confusion later
Always practicing with the topic tag visible	Mix in untagged, self-diagnosed practice — that's the actual transferable skill
Assuming binary search requires a sorted array	Check for ANY monotonic yes/no condition in the candidate answer
Not recognizing grid problems as graph problems	Ask: are cells nodes, and is adjacency up/down/left/right? If yes, the full graph toolkit applies
Confusing counting DP and optimization DP transitions	'Number of ways' sums over valid transitions; 'min/max cost' takes a min/max over them — different combine operations, same design process
Measuring progress by problems solved rather than recognition speed	Track how quickly you identify the RIGHT approach on fresh, untagged problems instead

Self-Check Questions

- Why do Hard LeetCode problems usually feel like several familiar ideas combined, rather than one entirely new idea?
- Why is practicing with tags visible not sufficient on its own to build real pattern recognition?
- Why doesn't an unsorted input array rule out binary search as a technique?
- Why are grid/matrix problems often better understood as graph problems?
- What's the practical difference between a counting DP transition and an optimization DP transition?

Cheat Sheet — Part 23

```
// Recognition checklist (Part 21's process, LeetCode-adapted)
// 1. Read the full description AND function signature
// 2. Check constraints -> target complexity (Part 21, Ch. 2)
// 3. Trace given examples by hand
// 4. Find the brute force, identify the bottleneck
// 5. Match the bottleneck's clue phrase to a pattern (table
//    above)
// 6. Verify complexity fits the constraints before implementing

// Study rhythm
// - Learn a technique WITH tags visible first
// - Then drill MIXED, UNTAGGED problems to test real
//    recognition
// - After solving: what clue phrase should have tipped me off?
// - Revisit old problems untimed, later, to check retention
```

Coming up next

Interview preparation (conceptual C++/STL/DSA questions with reasoned answers) and mock contests are still ahead in the roadmap. Say which you'd like built next and under what part number.